Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#06)

Use of Venus Simulator for RISC-V Procedure

Execution

Student Name: Muhammad Hamza Roll No: 24i-6513 Section: B

Date performed: 30 , Sep , 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:
// Write description of the task here ☟ (max. five to seven lines) This program demonstrates how to call a procedure without changing an important saved value. The value 2500 is stored in s0, while the procedure uses temporary registers to calculate 1000 + 3 = 1003. After returning, the program adds this result to the value already stored in s0 and displays the final answer.
Code:
.text main: li a0, 1 li s0, 2500 jal ra, procedure # call procedure add t1, s0, a0 # result should be 1003+2500 li a0,1 # setup system call for display (command in a0) mv a1, t1 # data to display in a1 ecall # displays result on console # we need to exit the program before the procedure run again # exit call of program li a0 , 10 ecall procedure: li t2, 3 # the answer is not correct beacause we overwrite the value of s0 register and now s0 become 3 instead of 2500 # we have to use another register for this operation # we use not t2 register for it li t0, 1000 add a0, t2, t0 ret
Output:


3503

Task 2:

Description:
// Write description of the task here 👇 (max. five to seven lines) The purpose of this code is to use the stack memory and to familiar with stack in this specific example what we do store s0 , t1 , t0 register on the stack. Before storing we allocate memory on stack by subtracting a offset from the current stack pointer because stack grows downward then we store the register and we also have to release the stack memory when we are done with it or we recovered the data from stack by adding the offset to stack pointer we will release the stack memory.
Code:
// Paste source code here 👇 .text main: addi s0, zero, 5 addi t1, zero, 10 addi t0, zero,15 addi sp, sp, -12 sw s0, 0(sp) sw t0, 4(sp) sw t1, 8(sp) lw t4, 8(sp) lw t2, 4(sp) lw t3, 0(sp) addi sp, sp, 12 # end call with no errors li a0 , 10 ecall
Output:
// Import output here 👇

Task 3:

Description:

// Write description of the task here ☟ (max. five to seven lines)

This program calculates the expression **(**a+b)(a-b). The two values a and b are passed using registers a0 and a1. The function first calculates their sum and difference, then multiplies these two results. The final answer is stored in s0 and displayed on the screen.

Code:

```
.text
main:
   li s0 , 0 # let s0 = x



   # we will use the argument register to give parameters to the functon
   li a0 , 10
   li a1 , 20


   jal ra , EqSolver


   li a0 , 1  #we will store 1 in a0 for displaying integer value which is stored in a1
   mv a1 , s0
   ecall

   # exit call to terminate the program
   li a0 , 10
   ecall

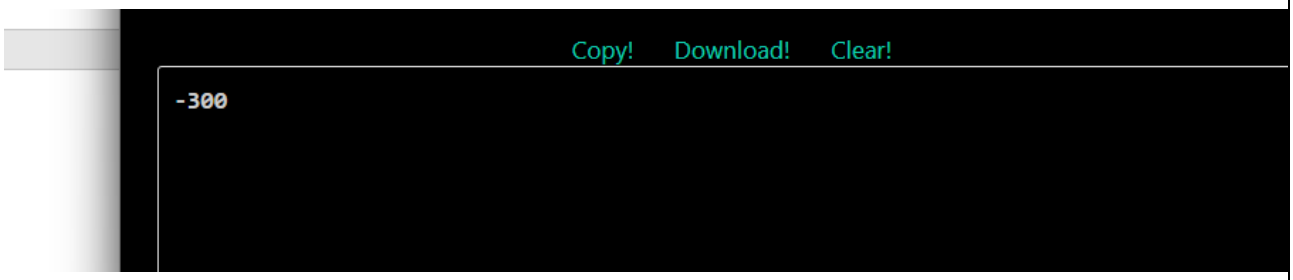

EqSolver:
   add t0 , a0 , a1 # a+b stored in t0
   sub t1 , a0 , a1 # a-b stored in t1

   mul s0 , t0 , t1 # multiplying t0 , t1 and storing it in the s0 save register

   ret
```

Output:

// Import output here



Task 4:

Description:
// Write description of the task here (max. five to seven lines) This program calculates the area and perimeter of a square. The side length is passed using a0. The Area function calculates side × side and stores the result in s0, while the Perimeter function calculates 4 × side and stores it in s1.
Code:

```
.text

main:

    # let side be a0
    li a0, 5

    # let area be s0
    li s0, 0

    # let perimeter be s1
    li s1, 0

    # Calling Area procedure
    jal ra, Area

    # Calling Perimeter procedure
    jal ra, Perimeter

    # Display Area
    li a0, 1
    mv a1, s0
```

```
    ecall

    # New line
    li a0, 11
    li a1, 10
    ecall

    # Display Perimeter
    li a0, 1
    mv a1, s1
    ecall

exit:
    li a0, 10
    ecall


Area:
    # Area = side * side
    mul t0, a0, a0
    mv s0, t0

    ret


Perimeter:
    # Perimeter = 4 * side
    li t0, 4
    mul t1, a0, t0
    mv s1, t1

    ret
```

Output:

// Import output here 👇



Task 5:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program solves the equation (x^3 + 5x^2 + 6x) for a given value of x. The value of x is passed to the EqSolver function using a0. The function calculates each part of the equation, adds them together, and stores the final result in s0.

Code:

```
.text
main:
   li s0 , 0 # let s0 = y



   # we will use the argument register to give parameters to the functon
   li a0 , 10 # x = 10


   jal ra , EqSolver


   li a0 , 1  #we will store 1 in a0 for displaying integer value which is stored in a1
   mv a1 , s0
   ecall

   # exit call to terminate the program
   li a0 , 10
   ecall


EqSolver:
   mv t0 , a0  # moving a copy of a0 in t0

   # now taking the power of x^3
   mul t1 , t0 , t0 # t1 = t0*t0
   mul t0 , t1 , t0 # t0 = (t0* t0) * t0 = t0^3

   # since we have t0^2(x^2)
   # we have to find the 5x^2
   # first we have to load the 5 into some register
   li t2 , 5
   mul t2 , t2 , t1  # t2 = 5x^2

   add t0 , t2 , t0  # t0 = x^3 + 5x^2

   # now we have to add 6 x

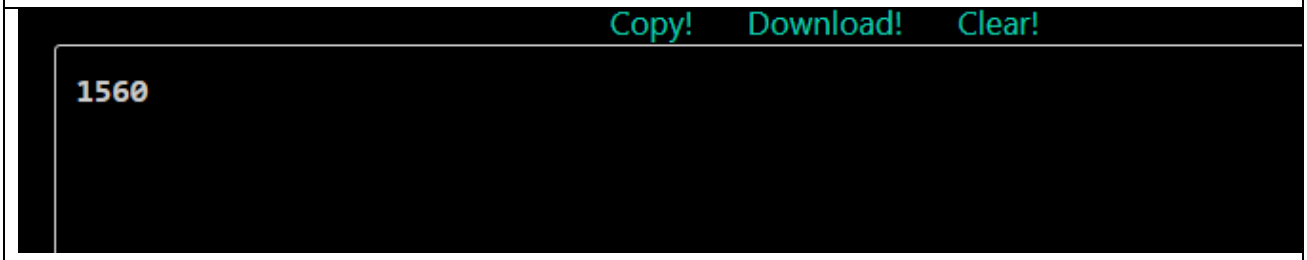
   addi t2 , x0 , 6
   mul t2 , t2 , a0 # t2 = 6 * x

   add t0 , t0 , t2

   mv s0 , t0  # saving the t2 value in s0


   ret
```

Output:



// Import output here 👇

Task 6:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program checks the values of n and k and calls one of three functions. If k = 1, it calls F1. If n > k, it calls F2. Otherwise, it calls F3. Each function performs a different calculation and stores the final answer in s0. At the end, the value stored in s0 is displayed on the screen.

Code:

```
.text
main:
    # let n and k are a0 , a1
    li a0 , 8
    li a1 , 10

    # let y is s0 register
    li s0 , 0

    li t0 , 1
    bne t0 , a1 , skiplabel1
        jal ra , F1
        j Display
    skiplabel1:
        ble a0 , a1 , skiplabel2
        jal ra , F2
```

```
    j Display

skiplabel2:
    jal ra, F3
    # no need to jump to display because the next instruction is dispalying



#Displaying the final value
Display:
    li a0 , 1
    mv a1 , s0
    ecall

exit:
    li a0 , 10
    ecall
F1:

    # since we have two argument a0 , a1

    mul t0 , a0 ,a0    # t0 = x^2
    mul t1 , a1 , a1   # t1 = z^2 = k^2
    add s0 , t0, t1 # storing the final value in s0(y)

    ret

F2:
    # here z = n = a0
    mul t0 , a0 , a0  # t0 = a0 * a0
    mul s0 , t0 , a0  # t0 = (a0 * a0) * a0 = a0 ^3 = n ^ 3  also directly storing on s0

    ret

F3:

    # here w = n = a0
    mv t0 , a0

    addi s0 , t0 , 5

    ret
```

Output:

// Import output here 👇

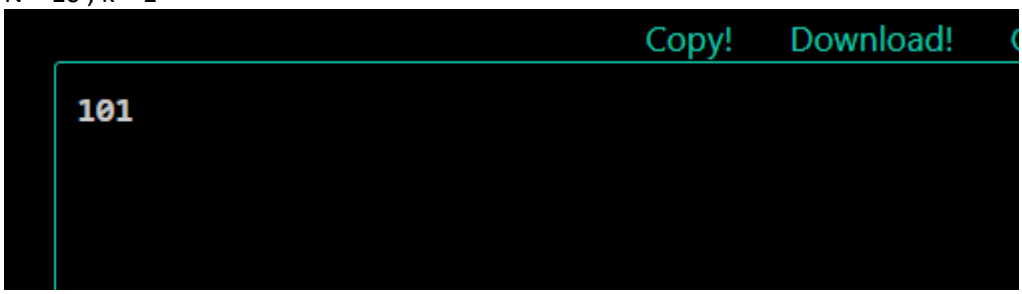
N = 8 , k = 10

```
Copy!  Download!  Clear!
13
```

N= 10 , k = 8



N = 10 , k = 1



Task 7:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program checks the values of n and k and selects one of three functions. If k = 0, it calls fun1. If n = k, it calls fun2. Otherwise, it calls fun3. The selected function calculates a value, stores the result in s0, and finally the program displays that result on the screen.

Code:

```
.text

main:

    # let n and k are a0, a1

    li a0, 10
    li a1, 0

    # let y be s0 register
    li s0, 0

    # if (k == 0)
    li t0, 0
    bne a1, t0, skiplabel1

        jal ra, fun1
```

```
        j Display

skiplabel1:

    # else if (n == k)
    bne a0, a1, skiplabel2

        jal ra, fun2
        j Display

skiplabel2:

    # else
    jal ra, fun3

    # Displaying the final value

Display:
    li a0, 1
    mv a1, s0
    ecall

exit:
    li a0, 10
    ecall


fun1:
    # int fun1(int x)
    # return (x * 2)
    mul t0, a0, a0
    mv s0, t0

    ret


fun2:
    # int fun2(int z)
    # return (z * 3)

    mul t0, a0,a0
    mul t0, t0 , a0

    mv s0, t0

    ret


fun3:
    # int fun3(int w)
    # return (w + 1)

    addi s0, a0, 1

    ret
```

Output:

// Import output here

```
li a0, 10
li a1, 0
```

```
100
```

```
li a0, 10
li a1, 10
```

```
1000
```

```
li a0, 8
li a1, 10
```

```
9
```

Task 8:

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program finds the Fibonacci number using a loop. We give the function a number, such as 6. It starts with the first two Fibonacci values, 0 and 1, and repeatedly adds them to find the next number. This continues until it reaches the required position. The final Fibonacci value is stored in s0 and then displayed on the screen.

Code:

```
.text
main:

   # lets  s0 is y
   li s0 , 0


   # intializing argument of the function
   li a0 ,6


   jal ra , fib

   #Displaying the final value
   Display:
      li a0 , 1
      mv a1 , s0
      ecall

   exit:
      li a0 , 10
      ecall



fib:
   # a= 0 , b= 1 , c= 1
   li t0 , 0
   li t1 , 1
   li t2 , 1

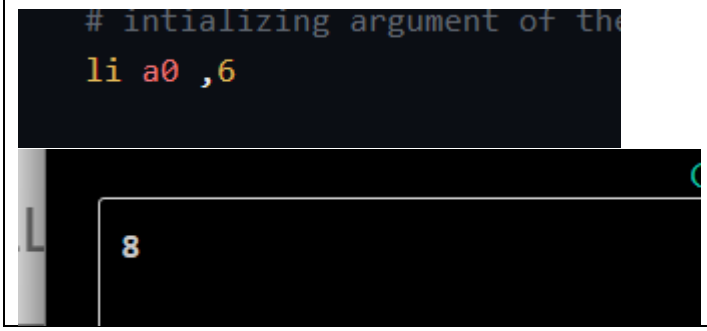
   beq a0 , x0 , return_a
      li t4 , 2 # i = 2
      for:
         bgt t4 , a0 , return
         add t2 , t0 , t1 # c = a+b
         mv t0 , t1  # a= b
         mv t1 , t2  # b = c

         addi t4 , t4 , 1
         j for
      return:
         mv s0 , t1
         ret
   return_a:
```

```
    mv s0 , t0
    ret
```

Output:

// Import output here 👇



Task 8: Recursive

Description:

// Write description of the task here 👇 (max. five to seven lines)

This program finds a Fibonacci number using recursion. We give it a number, for example 6. The function keeps calling itself with smaller numbers until it reaches 0 or 1. Then it starts returning back and adds the previous two Fibonacci numbers together. The stack is used to safely remember values and return addresses during these function calls. Finally, the answer is displayed on the screen.

Code:

```
.text
main:
   # n = 6
   li a0, 6

   # Call fib(6)
   jal ra, fib

   # Result is returned in a0
   mv s0, a0

Display:
   li a0, 1
   mv a1, s0
   ecall

exit:
   li a0, 10
   ecall
```

```
fib:

    li t0, 1
    ble a0, t0, return

    # Save ra and n
    addi sp, sp, -12
    sw ra, 0(sp)
    sw a0, 4(sp)


    # fib(n - 1)
    addi a0, a0, -1
    jal ra, fib

    # Save fib(n - 1)
    sw a0, 8(sp)


    # Restore original n
    lw a0, 4(sp)

    # fib(n - 2)
    addi a0, a0, -2
    jal ra, fib

    # Load fib(n - 1)
    lw t0, 8(sp)

    # fib(n) = fib(n-1) + fib(n-2)
    add a0, t0, a0

    # Restore return address
    lw ra, 0(sp)
    addi sp, sp, 12

return:
    ret
```

Output:

// Import output here 👇

